

PRO CREATOR – PRODUCTION ENGINEERING SPECIFICATION + REPOSITORY SCAFFOLD + STARTER CODE

Author: William Dickson
Primary Engine: macOS Big Sur 11.7.10
Companion: Android (Samsung Galaxy A17)
Architecture: Modular AI Orchestration Platform

```
=====
PRODUCTION-LEVEL                                ENGINEERING
=====
```

PART 1 — SPECIFICATIONS

1. SYSTEM ARCHITECTURE (PRODUCTION GRADE)

1.1 Architectural Pattern

Pro Creator follows a layered modular architecture:

Presentation Layer (Desktop UI / Android App) ↓ API Gateway Layer (FastAPI) ↓ Orchestration Layer (Workflow Engine) ↓ AI Service Layer (Script, Voice, Image, Video) ↓ Media Processing Layer (FFmpeg / Audio Tools) ↓ Storage Layer (Local FS + Metadata DB)

1.2 Core Design Principles

- Model-agnostic (swap models without rewriting system)
- Service-isolated modules
- Async task execution
- Project-based storage isolation
- Hardware-aware execution (Mac CPU optimized)
- Extendable to cloud rendering later

2. BACKEND ENGINEERING SPECIFICATION

2.1 Runtime Environment

- Python 3.10+
- FastAPI
- Uvicorn (ASGI server)
- Pydantic (data validation)
- SQLAlchemy or SQLite (metadata DB)
- FFmpeg (installed on macOS)
- Ollama (for LLMs)

2.2 Micro-Modules (Internal Services)

Each engine is its own service module.

Script Engine

Responsibilities: - Generate long-form scripts - Break into scenes - Generate titles and hooks

Input: { topic: string, duration_minutes: int, tone: string }

Output: { full_script: string, scenes: [{ id: int, text: string }] }

Voice Engine

Responsibilities: - Clone voice - Generate narration per scene - Return WAV file path

Input: { text: string, voice_profile: string }

Output: { audio_path: string, duration_seconds: float }

Image Engine

Responsibilities: - Generate scene image - Generate thumbnail

Input: { prompt: string, style: string }

Output: { image_path: string }

Video Engine

Responsibilities: - Sync narration + images - Add transitions - Add subtitles - Export MP4

Input: { project_id: string }

Output: { video_path: string }

3. STORAGE ARCHITECTURE

3.1 Directory Structure per Project

projects/ project_id/ script.txt scenes.json audio/ images/ video/ thumbnail.png

3.2 Metadata Database

SQLite Schema:

Project - id - title - topic - status - created_at

Scene - id - project_id - text - image_path - audio_path

4. SECURITY & PERFORMANCE

- Token-based local authentication
- Async endpoints
- BackgroundTasks for heavy processing
- Limit simultaneous render jobs
- Log file system

```
===== PART 2 — GITHUB-
READY                                REPOSITORY                SCAFFOLD
=====
```

ROOT STRUCTURE

```
pro_creator/ | | | | backend/ | | | | app/ | | | | main.py | | | | config.py | | | |
database.py | | | | models.py | | | | schemas.py | | | | routers/ | | | |
project.py | | | | script.py | | | | voice.py | | | | image.py | | | |
video.py | | | | services/ | | | | script_engine.py | | | | voice_engine.py | | | |
| | | | image_engine.py | | | | video_engine.py | | | | media_utils.py | | | |
| | | | file_manager.py | | | | logger.py | | | | requirements.txt | | | | run.sh |
| | | | frontend/ | | | | android_companion/ | | | | projects/ | | | | README.md
```

requirements.txt

fastapi uvicorn pydantic sqlalchemy python-multipart ffmpeg-python requests

```
===== PART 3 — FIRST
EXECUTABLE                                BACKEND                STARTER                CODE
=====
```

Below is a minimal but production-structured backend foundation.

backend/app/main.py

```
from fastapi import FastAPI
from app.routers import project
```

```
app = FastAPI(title="Pro Creator API")
```

```
app.include_router(project.router)
```

```
@app.get("/") def root(): return {"message": "Pro Creator Backend Running"}
```

backend/app/routers/project.py

```
from fastapi import APIRouter
from uuid import uuid4
import os
```

```
router = APIRouter(prefix="/project", tags=["Project"])
```

```
BASE_DIR = "projects"
```

```
@router.post("/create")
def create_project(title: str, topic: str):
    project_id = str(uuid4())
    project_path = os.path.join(BASE_DIR, project_id)
```

```
    os.makedirs(project_path, exist_ok=True)
    os.makedirs(os.path.join(project_path, "audio"), exist_ok=True)
    os.makedirs(os.path.join(project_path, "images"), exist_ok=True)
    os.makedirs(os.path.join(project_path, "video"), exist_ok=True)

    with open(os.path.join(project_path, "script.txt"), "w") as f:
        f.write("")

    return {
        "project_id": project_id,
        "message": "Project created successfully"
    }
```

backend/run.sh

#!/bin/bash

```
uvicorn app.main:app --reload --port 8000
```

===== HOW TO RUN (MAC BIG SUR) =====

1. Install Python 3.10+
2. Install FFmpeg via Homebrew
3. cd backend
4. pip install -r requirements.txt
5. chmod +x run.sh
6. ./run.sh
7. Open browser: <http://127.0.0.1:8000/docs>

You now have a working API foundation.

===== NEXT ENGINEERING STEPS =====

- Step 1: Integrate Ollama into script_engine.py
- Step 2: Integrate Coqui XTTS into voice_engine.py
- Step 3: Implement FFmpeg stitching in video_engine.py
- Step 4: Build React or Electron UI
- Step 5: Build Android companion app

===== FINAL NOTE
=====

This is now a production-oriented architecture, repository scaffold, and executable backend foundation for Pro Creator.

From here, development becomes systematic and scalable.